

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»
Навчально-науковий фізико-технічний інститут
Кафедра інформаційної безпеки

ЗВІТ

з дисципліни «Проектування високонавантажених систем»

Лабораторна робота № 3. Лічильник на Hazelcast: розподілена мапа і IAAtomicLong з CP Subsystem

Виконав: Вовкотруб Олександр Віталійович

Група: ФБ-61мн

Перевірив: Родіонов А. М.

Київ 2026

Зміст

1	Мета роботи	2
2	Постановка задачі	2
3	Хід виконання	2
3.1	Середовище	2
3.2	Розгортання кластера	3
3.3	Власний клієнт Hazelcast для Swift	4
3.4	Частина I: чотири способи інкременту	5
3.5	Частина II: веб-лічильник на IAtomicLong	6
3.6	Методика вимірювань	6
3.7	Аналітичний ноутбук	6
4	Результати	7
4.1	Частина I: чотири способи інкременту	7
4.2	Навіщо лічильнику IAtomicLong і CP Subsystem	8
4.3	Частина II: порівняння сховищ веб-лічильника	9
4.4	Додаткове дослідження: підключення до лідера CP-групи	10
4.5	Логи вузлів	11
5	Висновки	12
6	Джерела	13

1 Мета роботи

Розгорнути кластер Hazelcast із трьох вузлів і на прикладі спільного лічильника порівняти способи конкурентного оновлення даних у ньому: розподілену мапу IMap без блокувань, з песимістичним і з оптимістичним блокуванням, а також IAtomicLong у CP Subsystem. Оцінити коректність і швидкодію кожного способу. Використати IAtomicLong як сховище веб-лічильника з лабораторної роботи № 1 і порівняти його зі сховищами в пам'яті, на диску і в PostgreSQL.

2 Постановка задачі

Робота складається з двох частин.

Частина I. Підняти кластер Hazelcast 5.4.0 із трьох вузлів. Збільшувати один спільний лічильник із 10 одночасних задач, по 10 000 разів кожна, тобто очікуване фінальне значення дорівнює 100 000. Зробити це чотирма способами:

1. IMap без блокувань: get, збільшення на клієнті, put;
2. IMap з песимістичним блокуванням: lock, get, put, unlock;
3. IMap з оптимістичним блокуванням: get, потім replace(key, old, old + 1) з повтором у разі невдачі;
4. IAtomicLong.incrementAndGet() у CP Subsystem.

Для кожного способу виміряти час, фінальне значення і кількість повторів. Пояснити, чому одні способи втрачають оновлення, а інші ні, і навіщо лічильнику CP Subsystem.

Частина II. Використати IAtomicLong як сховище веб-лічильника з роботи № 1 (GET /inc, GET /count). Навантажити його 10 клієнтами по 10 000 запитів і звести в одну таблицю з результатами для сховищ у пам'яті, на диску (файл з fsync) і в PostgreSQL з робіт № 1 і № 2.

До звіту додати логи вузлів кластера, які підтверджують склад кластера і роботу CP Subsystem.

3 Хід виконання

3.1 Середовище

Усі процеси (три вузли Hazelcast, клієнти hz-bench, counter-server, loadgen) працюють на одній машині й спілкуються через loopback. Мережеві затримки тут мінімальні, а всі процеси ділять ті самі 16 потоків CPU. Версію 5.4.0 обрано свідомо: починаючи з 5.5 CP Subsystem доступний лише в Enterprise-редакції [5], а в 5.4.0 він працює повноцінно, на Raft із трьох членів.

Таблиця 3.1. Апаратне і програмне середовище

Параметр	Значення
CPU	Intel Coffee Lake (інженерний зразок, 2,60 ГГц), 8 ядер, 16 потоків, 0,8...4,4 ГГц, L3 16 MiB
RAM	62 GiB (під час вимірювань вільно близько 26 GiB)
Диск	NVMe SSD Samsung 970 EVO Plus 1 TB, ext4
ОС	Fedora Linux 44, ядро 7.2.5-200.fc44.x86_64
Мова	Swift 6.4, збірка -c release
Hazelcast	Hazelcast Platform 5.4.0 (20240415), OpenJDK 25.0.4.1, heap 512 МБ на вузол
Кластер	3 вузли 127.0.0.1:5701...5703, кластер dev, TCP-IP discovery
CP Subsystem	cp-member-count: 3, group-size: 3, без persistence
Імар	налаштування за замовчуванням, backup-count: 1

3.2 Розгортання кластера

Кожен вузол має власний файл конфігурації `deploy/hazelcast-node{1,2,3}.yaml`. Вузли відрізняються лише ім'ям і портом. Multicast і автовиявлення вимкнено, вузли знаходять один одного за явним списком адрес. CP Subsystem увімкнено на всіх трьох вузлах:

```
hazelcast:
  cluster-name: dev
  instance-name: hz-node1
  network:
    port:
      port: 5701
      auto-increment: false
    join:
      tcp-ip:
        enabled: true
        member-list:
          - 127.0.0.1:5701
          - 127.0.0.1:5702
          - 127.0.0.1:5703
  cp-subsystem:
    cp-member-count: 3
    group-size: 3
```

Скрипт `scripts/hz.sh start` запускає три вузли й чекає в їхніх логах рядок CP Group Members {... size:3 ...}. Він означає, що група METADATA має всіх трьох членів і CP

Subsystem готовий. Команда `scripts/hz.sh stop` зупиняє кластер.

3.3 Власний клієнт Hazelcast для Swift

Офіційного клієнта Hazelcast для Swift немає, тому для роботи написано власний клієнт на SwiftNIO [8] поверх Open Binary Client Protocol 2.x [3]. Код клієнта (1 936 рядків) лежить у каталозі `Sources/HazelcastClient/` і складається з п'яти шарів:

1. Протокол (Protocol/). Після з'єднання клієнт надсилає три байти CP2. Повідомлення складається з кадрів: довжина `int32`, прапорці `uint16`, далі вміст. Перший кадр несе тип повідомлення, `correlation id` і номер розділу. Кодування й декодування кадрів зроблено як `ChannelHandler` у конвеєрі SwiftNIO.
2. Сериалізація (Serialization/). Ключі й значення кодуються у формат Data Hazelcast. Номер розділу ключа обчислюється як `MurmurHash3 x86_32` від серіалізованого ключа за модулем кількості розділів, яку вузол повідомляє під час автентифікації.
3. Кодеки (Codecs/). Один тип на операцію: автентифікація, `Map.Get`, `Map.Put`, `Map.Lock`, `Map.Unlock`, `Map.ReplaceIfSame`, `CPGroup.CreateCPGroup`, операції `AtomicLong`. Ідентифікатори типів і порядок полів звірено з описом протоколу для версії 5.4.
4. З'єднання (Connection/). Одне з'єднання з одним вузлом (`unisocket`). Запити багатьох задач мультиплексуються в ньому, а відповідь знаходить свого адресата за `correlation id` (`InvocationRegistry`). Якщо розділ або лідер CP-групи живе на іншому вузлі, вузол сам пересилає йому операцію.
5. API (API/). `HazelcastClient`, `HazelcastMap`, `CPSubsystem`, `AtomicLong`. Блокування `IMap` належить парі «клієнт, thread id», як потоку Java, тому кожна з 10 задач має власний thread id.

Операції з мапою адресуються розділу, якому належить ключ:

```
func invoke(_ request: ClientMessage, on key: HazelcastData,
            expecting responseType: Int32) async throws -> ClientMessage {
    var request = request
    request.partitionID = key.partitionID(partitionCount: partitionCount)
    return try await connection.invoke(request, expecting: responseType)
}
```

Інкремент `IAAtomicLong` це один запит `AtomicLong.AddAndGet` до СР-групи (файл `API/AtomicLong.swift`):

```
public func addAndGet(_ delta: Int64) async throws -> Int64 {
    let request = AtomicLongAddAndGetCodec.encodeRequest(
        groupID: groupID, name: name, delta: delta)
    let response = try await client.invoke(
        request, expecting: AtomicLongAddAndGetCodec.responseType)
    return try AtomicLongAddAndGetCodec.decodeResponse(response)
}
```

Клієнт покрито тестами в каталозі `Tests/HazelcastClientTests/`. Вони перевіряють кодування кадрів, серіалізацію, байтовий вміст запитів кодеків і роботу з живим кластером.

3.4 Частина I: чотири способи інкременту

Програма `hz-bench` (`Sources/hz-bench/`) скидає лічильник у 0, запускає 10 задач у `withThrowingTaskGroup` і вимірює `ContinuousClock` лише фазу інкрементів. Потім вона читає фінальне значення й друкує рядок CSV. Способи інкременту описано в `Sources/hz-bench/Variant.swift`. Варіант без блокувань і варіант з `IAAtomicLong`:

```
case (.mapNoLock, .map(let map, let key)):
    let value = try await map.get(key, threadID: threadID) ?? 0
    try await map.put(key, value + 1, threadID: threadID)
case (.atomicLong, .atomicLong(let counter)):
    try await counter.incrementAndGet()
```

Песимістичний і оптимістичний варіанти [2]:

```
case (.mapPessimistic, .map(let map, let key)):
    try await map.withLock(key, threadID: threadID) {
        let value = try await map.get(key, threadID: threadID) ?? 0
        try await map.put(key, value + 1, threadID: threadID)
    }
case (.mapOptimistic, .map(let map, let key)):
    var retries = 0
    while true {
        let value = try await map.get(key, threadID: threadID) ?? 0
        if try await map.replace(key, expected: value, new: value + 1,
            threadID: threadID) { return retries }
        retries += 1
    }
```

withLock знімає блокування і тоді, коли тіло завершилося помилкою. replace це compare-and-set на вузлі, якому належить ключ: він повертає false, якщо значення вже змінилося, і спроба починається спочатку.

3.5 Частина II: веб-лічильник на IAtomicLong

Сховище HazelcastCounterStore (Sources/CounterCore/) реалізує той самий протокол CounterStore, що й сховища з робіт № 1 і № 2. GET /inc викликає IAtomicLong("web-counter").incrementAndGet() [6], GET /count викликає get(). Усі HTTP-запити йдуть через одне спільне з'єднання клієнта з вузлом 5701. Сервер запускається так:

```
counter-server --store hazelcast --hz-address 127.0.0.1:5701
```

3.6 Методика вимірювань

1. scripts/hz.sh start запускає кластер і чекає на CP Subsystem.
2. scripts/task3.sh збирає hz-bench і тричі проганяє всі чотири варіанти, 10 задач по 10 000 інкрементів. Результат записується в reports/data/task3-bench.csv.
3. Той самий скрипт запускає counter-server --store hazelcast і loadgen з 10 клієнтами по 10 000 запитів, тричі. Результат записується в reports/data/task3-web.csv. Числа для інших сховищ узято з reports/data/task1.csv (рядки з 10 клієнтами) і reports/data/task2-web.csv.
4. У таблицях наведено медіану трьох повторів, мінімум і максимум. Обидві частини пройшли на одному запуску кластера 02.10.2026 з 18:52 до 18:58. Інші бенчмарки в цей час не працювали.

```
scripts/hz.sh start
scripts/task3.sh # task3-bench.csv, task3-web.csv
scripts/hz.sh stop
```

3.7 Аналітичний ноутбук

Обробку даних винесено в ноутбук solution.ipynb, опублікований разом зі звітом. Ноутбук читає task3-bench.csv, task3-web.csv, task2-web.csv і task1.csv, рахує в pandas медіани й діапазони для таблиць 4.1...4.3, розбирає логи вузлів Hazelcast і будує рисунки 4.1 і 4.2. Розділ «Жива перевірка» при RUN_LIVE = True запускає тести кодування кадрів клієнта, піднімає кластер із трьох вузлів, проганяє hz-bench для map-no-lock і atomic-long на 10×300 інкрементах і зупиняє кластер. Це демонстрація на зменшеному навантаженні, а не джерело чисел звіту.

4 Результати

4.1 Частина I: чотири способи інкременту

Таблиця 4.1. Час і пропускна здатність, 10 задач по 10 000 інкрементів (медіана трьох повторів)

Варіант	Медіана, с	Мін., с	Макс., с	Інкр./с	Втрачено
map-nolock	5,14	5,08	5,98	19 456,5	79 737
map-pessimistic	59,05	54,19	60,53	1 693,5	0
map-optimistic	27,41	22,66	41,30	3 648,8	0
atomic-long	4,27	4,22	4,34	23 431,6	0

Таблиця 4.2. Фінальні значення і кількість повторів у кожному з трьох прогонів (очікується 100 000)

Варіант	Фінальне значення	Повтори
map-nolock	20 162; 20 263; 20 265	0; 0; 0
map-pessimistic	100 000; 100 000; 100 000	0; 0; 0
map-optimistic	100 000; 100 000; 100 000	455 817; 457 866; 453 715
atomic-long	100 000; 100 000; 100 000	0; 0; 0

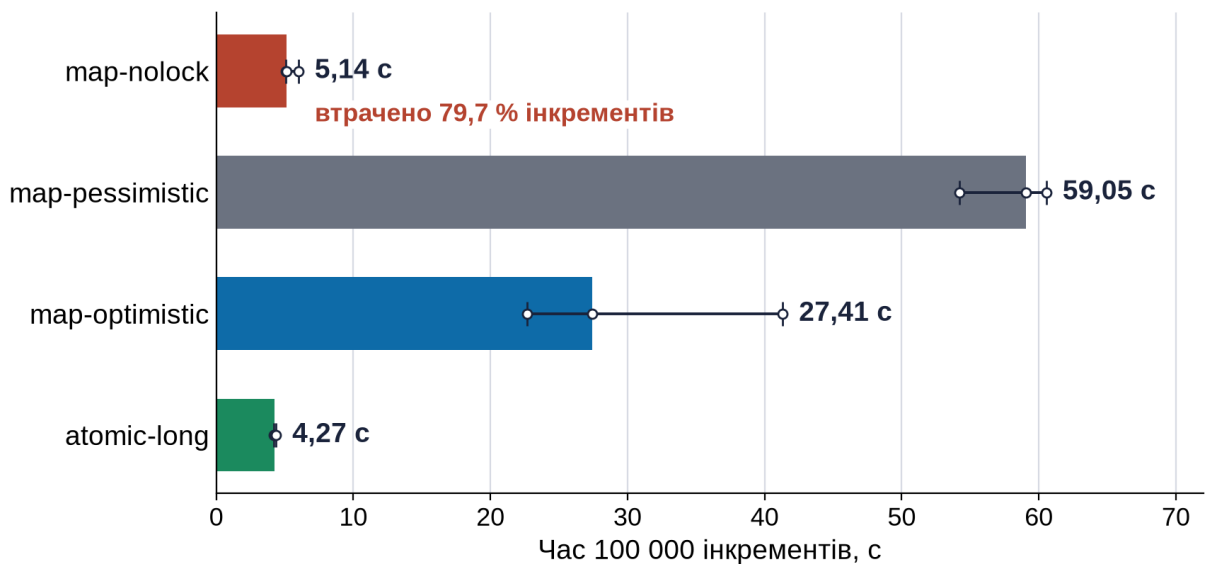


Рисунок 4.1. Час 100 000 інкрементів: медіана трьох повторів (стовпці), мінімум і максимум (відмітки), окремі повторення (точки)

map-nolock втрачає 79,7 % інкрементів (медіана 79 737 з 100 000). Кожна з операцій get і put атомарна на вузлі-власнику розділу, а пара разом ні. Між get і put однієї задачі інші задачі читають те саме старе значення v і потім записують те саме $v + 1$. З кількох одночасних інкрементів лишається один. Це та сама аномалія lost update, що й у PostgreSQL у роботі № 2, тільки в in-memory data grid. Варіант швидкий (5,14 с), але результат неправильний.

map-pessimistic дає рівно 100 000, але працює найдовше: 59,05 с, у 13,8 раза довше за atomic-long. Кожен інкремент коштує чотири запити до кластера (lock, get, put, unlock), а критичні секції 10 задач виконуються строго по черзі. На один інкремент припадає 0,59 мс, і паралелізм задач нічого не дає. Повторів немає: задача без блокування просто чекає.

map-optimistic теж дає рівно 100 000 і працює приблизно вдвічі швидше за песимістичний варіант (27,41 с). Ціна в повторях: за медіаною 455 817, тобто 4,56 невдалої спроби на кожен успішний інкремент. Задачі не чекають одна одну, і кожна невдача коштує лише двох коротких запитів. Розкид часу тут найбільший (від 22,66 до 41,30 с), бо кількість колізій залежить від того, як задачі переплітаються в часі. Найдовший прогін (41,30 с) збігся із записом HealthMonitor на всіх трьох вузлах о 18:57:19 про високе навантаження системи, а кількість повторів у ньому майже та сама (453 715). Причина, найімовірніше, у загальному навантаженні хоста.

atomic-long коректний і найшвидший: 4,27 с, 23 431,6 інкр./с. Він швидший навіть за неправильний map-no-lock (5,14 с), бо на інкремент потрібен один запит, а не два. Читання, зміна і запис відбуваються всередині CP-групи на сервері, тож вікна для гонки немає. Кожен інкремент проходить через Raft-лог групи default і підтверджується більшістю, тобто двома з трьох вузлів. Лог вузла 3 це підтверджує: після першого прогону лідерство групи default перейшло на нього при last log index: 100002, тобто 100 000 інкрементів лягли в лог окремими записами (плюс службові записи, зокрема set(0)).

4.2 Навіщо лічильнику IAtomicLong і CP Subsystem

Правильний результат дають три способи з чотирьох: IMap з lock, IMap з replace і IAtomicLong. Відмінність між ними у ціні та гарантіях.

Ціна. lock і replace переносять узгодження на клієнта: кілька запитів на інкремент і або черга, або повтори. IAtomicLong виконує інкремент атомарно на сервері за один запит. У наших вимірюваннях він у 13,8 раза швидший за lock і в 6,4 раза швидший за replace.

Гарантії. IMap належить до AP-частини Hazelcast [2]. Розділи реплікуються за схемою primary і backup без консенсусу, а IMap.lock не захищений від split-brain: під час розділення мережі обидві половини кластера можуть вважати себе власниками ключа. CP Subsystem побудований на алгоритмі Raft [1, 4] і дає лінеаризовані операції. Запис зараховується лише після підтвердження більшістю, тому меншість після розділення мережі запис не прийме і розбіжних значень лічильника не виникне. Група з трьох вузлів витримує втрату одного, тому CP Subsystem потребує щонайменше трьох членів (cp-member-count: 3).

Обмеження стенду: усі три вузли працюють на одній машині, тому стійкість до відмов тут описано, але не перевірено. CP Subsystem працює без persistence, Raft-лог зберігається в пам'яті вузлів.

4.3 Частина II: порівняння сховищ веб-лічильника

Таблиця 4.3. Веб-лічильник, 10 клієнтів по 10 000 запитів GET /inc (медіана трьох повторів, у всіх прогонах фінальне значення 100 000)

Сховище	Медіана, с	Мін., с	Макс., с	Запитів/с
In-memory	3,20	2,92	3,31	31 256,0
Disk (fsync)	818,08	814,26	824,12	122,2
PostgreSQL	302,44	199,76	306,17	330,6
Hazelcast (CP)	8,88	8,79	9,08	11 259,7

Дані для пам'яті й диска взято з reports/data/task1.csv, для PostgreSQL з reports/data/task2-web.csv, для Hazelcast з reports/data/task3-web.csv.

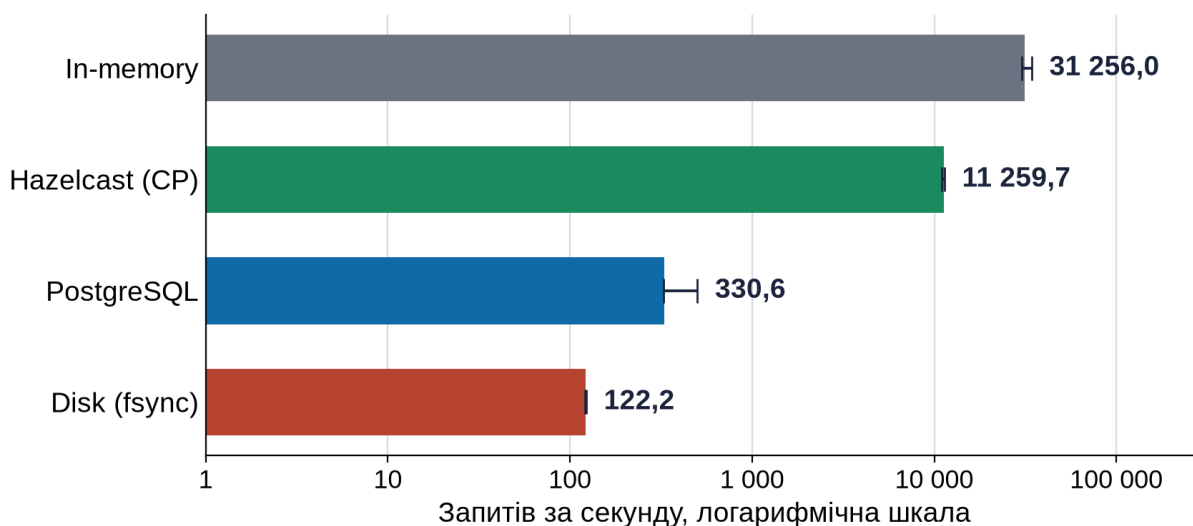


Рисунок 4.2. Пропускна здатність веб-лічильника для чотирьох сховищ, логарифмічна шкала: медіана трьох повторів (стовпці), мінімум і максимум (відрізки)

Hazelcast у 2,8 раза повільніший за лічильник у пам'яті (11 259,7 проти 31 256,0 запитів/с). У пам'яті інкремент це атомарна операція всередині процесу сервера. З Hazelcast на кожен HTTP-запит додаються запит до вузла 5701, пересилання лідеру CP-групи і Raft-реплікація на більшість вузлів. Зате значення не зникає разом із процесом counter-server, і кілька екземплярів сервера можуть рахувати в один лічильник.

Hazelcast у 34 рази швидший за PostgreSQL і в 92 рази швидший за файл з fsync. В обох цих сховищах кожен інкремент чекає на запис на диск: fsync WAL на кожен коміт у PostgreSQL і fsync файлу у файловому сховищі. Hazelcast забезпечує довговічність реплікацією в пам'ять інших вузлів, тому його межа це CPU і loopback, а не диск. Гарантії різні: PostgreSQL і файл переживуть вимкнення всієї машини, а кластер Hazelcast без persistence ні. Натомість кластер переживає втрату одного вузла з трьох.

Веб-варіант (11 259,7 запитів/с) повільніший за прямий hz-bench atomic-long (23 431,6 інкр./с) через накладні витрати HTTP: розбір запиту, відповідь і ще один мережевий

обмін на кожен інкремент. Крім того, loadgen і counter-server конкурують за ті самі ядра з трьома JVM. Розкид між повторами Hazelcast близько 3 % (від 8,79 до 9,08 с).

4.4 Додаткове дослідження: підключення до лідера CP-групи

Після основних вимірювань веб-лічильник оптимізовано, докладний опис наведено у файлі reports/throughput.md. Для Hazelcast змінено три речі: легший генератор навантаження (loadgen --engine nio), сервер на ChannelHandler без переходів між задачами (--engine handler) і вибір вузла, через який запити входять у кластер. Сховище й гарантії не змінювались: той самий IAtomicLong у CP-групі з трьох членів.

Вузол, який не є лідером групи default, пересилає кожен Raft-запит лідеру, а це зайвий мережевий хоп. Пряма маршрутизація CP-запитів до лідера є лише в Enterprise-клієнтах версії 5.5 і новіших [7], тому сервер з параметром --hz-address 127.0.0.1:5701, 127.0.0.1:5702, 127.0.0.1:5703 підключається до всіх трьох вузлів, вимірює медіану 300 послідовних get() на кожному і залишає найшвидше з'єднання.

Таблиця 4.4. hz-bench atomic-long через кожен вузол окремо (10 задач по 5 000 інкрементів, два повтори) і проба сервера

Вузол	Повтор 1, інкр./с	Повтор 2, інкр./с	Медіана get(), мкс
5701	14 834	21 636	312
5702 (лідер)	27 566	29 306	152
5703	20 870	20 515	257

У цьому окремому запуску кластера лідером групи default був вузол 5702, що підтверджує лог, і проба обрала саме його.

Таблиця 4.5. Веб-лічильник на Hazelcast після оптимізацій, 10 клієнтів по 10 000 запитів (медіана трьох почергових повторів)

Сервер	Вузол	loadgen	Запитів/с	Мін.	Макс.
базовий	5701	AsyncHttpClient	11 370	11 069	12 534
базовий	5701	nio	16 526	14 410	16 646
handler	5701	AsyncHttpClient	13 056	11 626	13 473
handler	5701	nio	17 328	15 406	17 757
базовий	лідер 5702	AsyncHttpClient	14 550	12 372	14 579
handler	лідер 5702	nio	20 367	18 105	21 046

Підключення до лідера дало +18 % (з 17 328 до 20 367 запитів/с), а всі кроки разом у 1,8 раза (з 11 370 до 20 367). Це близько до прямого hz-bench atomic-long через лідера (27 566 і 29 306 інкр./с). Залишок припадає на Raft-коміт на трьох JVM, які ділять ті самі ядра. Якщо лідерство переїде на інший вузол, коректність не постраждає, повернеться лише зайвий хоп.

4.5 Логи вузлів

Повні логи збережено в reports/logs/hz-node1.log, hz-node2.log, hz-node3.log (разом близько 130 КБ), лог веб-сервера в reports/logs/counter-server-hazelcast.log. Нижче наведено ключові уривки. Префікси рядків (дата, рівень, потік, адреса вузла) скорочено, довгі UUID обрізано до першого блоку.

Старт вузла 2: версія і CP Subsystem із трьох членів.

```
18:52:33,008 [c.h.system] Hazelcast Platform 5.4.0 (20240415)
    starting at [127.0.0.1]:5702
18:52:34,342 [c.h.c.CPSubsystem] CP Subsystem is enabled with 3 members.
```

Членство в кластері: три вузли (вузол 2).

```
Members {size:3, ver:3} [
  Member [127.0.0.1]:5701 - 89211daa-dca9-4cd9-ab96-01a4ee1c39ae
  Member [127.0.0.1]:5703 - 255cb7ad-4963-4017-a75b-6fcce9b43890
  Member [127.0.0.1]:5702 - 1d98c7c6-827f-485a-945b-f73dcf06362a this
]
```

Група METADATA: size:3 і вибори лідера (вузол 1).

```
18:52:42,792 VoteResponseHandlerTask(METADATA) Vote granted from
    RaftEndpoint{uuid='1d98c7c6...'} for term: 1
18:52:42,792 VoteResponseHandlerTask(METADATA) We are the LEADER!
CP Group Members {groupId: METADATA(0), size:3, term:1, logIndex:0} [
  CPMember{uuid=89211daa..., address=[127.0.0.1]:5701} - LEADER this
  CPMember{uuid=1d98c7c6..., address=[127.0.0.1]:5702}
  CPMember{uuid=255cb7ad..., address=[127.0.0.1]:5703}
]
18:52:43,598 [MetadataRaftGroupManager] CP Subsystem is initialized
    with: [5701, 5702, 5703]
```

Перше звернення до IAtomicLong створює CP-групу default з трьох вузлів (вузол 1).

```
18:54:32,514 [MetadataRaftGroupManager] New CPGroupId{name='default',
seed=0, groupId=3883} is created with [5702, 5701, 5703]
```

Передача лідерства в групі default після 100 000 інкрементів (вузол 3).

```
18:54:43,609 TriggerLeaderElectionHandlerTask(default) Starting a new
leader election since the current leader:
RaftEndpoint{uuid='89211daa...'} in term: 1 asked for
a leadership transfer!
18:54:43,612 LeaderElectionTask(default) Leader election started for
term: 2, last log index: 100002, last log term: 1
18:54:43,617 VoteResponseHandlerTask(default) We are the LEADER!
CP Group Members {groupId: default(3883), size:3, term:2, logIndex:0} [
  CPMember{uuid=1d98c7c6..., address=[127.0.0.1]:5702}
  CPMember{uuid=89211daa..., address=[127.0.0.1]:5701}
  CPMember{uuid=255cb7ad..., address=[127.0.0.1]:5703} - LEADER this
]
```

Зупинка кластера (вузол 3 виходить останнім).

```
Members {size:1, ver:5} [
  Member [127.0.0.1]:5703 - 255cb7ad-4963-4017-a75b-6fcce9b43890 this
]
18:58:54,743 [c.h.i.i.Node] Hazelcast Shutdown is completed in 62 ms.
```

5 Висновки

1. Кластер Hazelcast 5.4.0 з трьох вузлів і CP Subsystem на всіх трьох розгорнуто на одній машині. Логи підтверджують Members {size:3}, групи METADATA і default з трьох членів і вибори лідера за Raft. Для роботи з кластером написано власний клієнт на Swift і SwiftNIO поверх Open Binary Client Protocol 2.x.
2. Інкремент через IMap.get і IMap.put без синхронізації втратив 79,7 % оновлень: фінальне значення близько 20 200 замість 100 000. Атомарною має бути вся пара «прочитати, записати», а не кожна операція окремо.
3. IMap з песимістичним блокуванням коректний, але найповільніший: 59,05 с, чотири запити на інкремент і строга черга задач. Оптимістичне блокування через replace коректне і вдвічі швидше (27,41 с), але при 10 конкурентних задачах витрачає 4,56 повтору на кожен успішний інкремент.

4. IAtomicLong у CP Subsystem коректний і найшвидший: 4,27 с, 23 431,6 інкр./с, у 13,8 разів швидше за lock і в 6,4 разів за replace. Інкремент виконується на сервері одним запитом, а Raft на трьох вузлах дає лінеаризованість і стійкість до втрати одного вузла. Для лічильника це правильний примітив.
5. Як сховище веб-лічильника Hazelcast дав 11 259,7 запитів/с. Він поступається лише пам'яті (31 256,0, у 2,8 разів), але в 34 рази швидший за PostgreSQL (330,6) і в 92 рази швидший за файл з fsync (122,2), бо не чекає на диск. Підключення до лідера CP-групи разом з легшим клієнтом і сервером підняло результат до 20 367 запитів/с, з них +18 % дав саме лідер.

6 Джерела

1. Hazelcast Platform 5.4 Documentation. CP Subsystem [Електронний ресурс]. Режим доступу: <https://docs.hazelcast.com/hazelcast/5.4/cp-subsystem/cp-subsystem> (дата звернення: 05.10.2026).
2. Hazelcast Platform 5.4 Documentation. Locking Maps [Електронний ресурс]. Режим доступу: <https://docs.hazelcast.com/hazelcast/5.4/data-structures/locking-maps> (дата звернення: 05.10.2026).
3. Hazelcast Open Binary Client Protocol [Електронний ресурс]. Режим доступу: <https://github.com/hazelcast/hazelcast-client-protocol> (дата звернення: 05.10.2026).
4. Ongaro D., Ousterhout J. In Search of an Understandable Consensus Algorithm [Електронний ресурс]. Режим доступу: <https://raft.github.io/raft.pdf> (дата звернення: 05.10.2026).
5. Hazelcast. Changes to Community Edition [Електронний ресурс]. Режим доступу: <https://hazelcast.com/blog/changes-to-community-edition/> (дата звернення: 05.10.2026).
6. Hazelcast Platform 5.4 Documentation. IAtomicLong [Електронний ресурс]. Режим доступу: <https://docs.hazelcast.com/hazelcast/5.4/data-structures/iatomicleong> (дата звернення: 05.10.2026).
7. Hazelcast Platform 5.7 Documentation. Java Client [Електронний ресурс]. Режим доступу: <https://docs.hazelcast.com/hazelcast/5.7/clients/java> (дата звернення: 05.10.2026).
8. SwiftNIO [Електронний ресурс]. Режим доступу: <https://github.com/apple/swift-nio> (дата звернення: 05.10.2026).